`*this` is not locked by the calling thread.

Throws

Nothing.

D.5.4. std::recursive_timed_mutex class

The `std::recursive_timed_mutex` class provides support for locks with timeouts on top of the mutual exclusion and synchronization facility provided by `std::recursive_mutex`. Prior to accessing the data protected by the mutex, the mutex must be *locked* by calling `lock()`, `try_lock()`, `try_lock_for()`, or `try_lock_until()`. If a lock is already held by another thread, an attempt to acquire the lock will fail (`try_lock()`), block until the lock can be acquired (`lock()`), or block until the lock can be acquired or the lock attempt times out (`try_lock_for()` or `try_lock_until()`). Once a lock has been acquired (whichever function was used to acquire it) it must be released by calling `unlock()` before another thread can acquire the lock on the mutex.

This mutex is *recursive*, so a thread that holds a lock on a particular instance of `std::recursive_timed_mutex` may acquire additional locks on that instance through any of the lock functions. All of these locks must be released by a corresponding call to `unlock()` before another thread can acquire a lock on that instance.

`std::recursive_timed_mutex` meets the `TimedLockable` requirements.

Class definition

```
class recursive_timed_mutex
{
public:
    recursive_timed_mutex(recursive_timed_mutex const&)=delete;
    recursive_timed_mutex& operator=(recursive_timed_mutex
```